

One image. Many layers of understanding.

Step inside **LeNet**, **AlexNet** and **VGG-16**. See the network, follow its features, trace the dimensions, then count what it learns.

01 / SEE The architecture How the layers connect.	02 / FOLLOW The feature maps What happens to an image.	03 / TRACE The dimensions How big is each output?	04 / COUNT The parameters How many values are learned?
--	---	--	---

CHAPTER 1 / FOLLOW A DIGIT

A handwritten digit → 10 classes

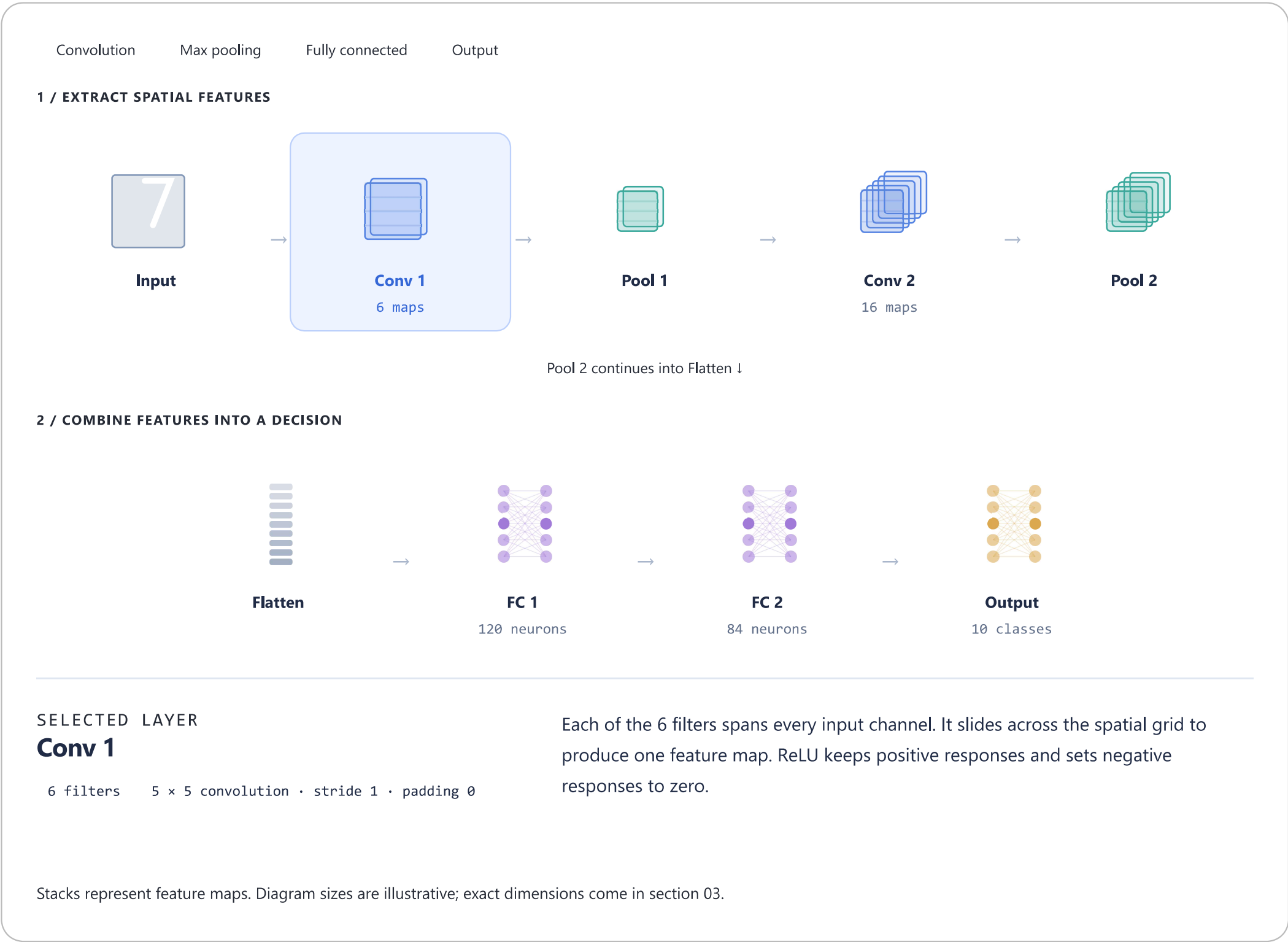
LeNet

Begin with a small grayscale image. Two rounds of convolution and pooling extract features; fully connected layers turn them into a digit prediction.

01 / SEE THE WHOLE NETWORK

Start with the big picture.

Read each row from left to right, then continue on the next row. Select a layer to inspect what it does.



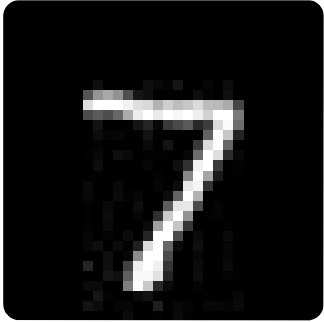
02 / FOLLOW THE IMAGE

Pixels become responses.

Keep the same input and step through the network. Every map below is an activation from a trained network, not a resized copy of the input.

Input image

32 × 32 × 1



→

Conv 1

28 × 28 × 6



Channel 1



Channel 2



Channel 3



Channel 4



Channel 5



Channel 6

Low response

High response

· Each map is scaled separately to reveal its pattern.

Conv 1: Each of the 6 filters spans every input channel. It slides across the spatial grid to produce one feature map. ReLU keeps positive responses and sets negative responses to zero.

A channel answers “where did this filter respond?” Early maps may highlight strokes, edges or contrast. Deeper maps combine earlier responses; they need not look like a recognizable picture.

After pooling, do the channels get merged into one image?

03 / TRACE THE DIMENSIONS

Trace the dimensions of the image as it flows through the network.

Open a layer, substitute the given values, then fill in the intermediate results and final output shape. Dimensions are for one image.

$$\text{Output} = \frac{\text{Input} - K + 2(P)}{\text{Stride}} + 1$$

K is the kernel size. P is the padding.

LAYER	OPERATION	OUTPUT SHAPE
Conv 1	5 × 5 convolution · stride 1 · padding 0	? × ? × ? Fill the steps +
Pool 1	2 × 2 max pool · stride 2 · padding 0	? × ? × ? Fill the steps +
Conv 2	5 × 5 convolution · stride 1 · padding 0	? × ? × ? Fill the steps +

https://kartikgupta98.github.io/dl-worksheets/worksheet10_architectures.html#lenet

3/17

Pool 2	2 × 2 max pool · stride 2 · padding 0	? × ? × ? Fill the steps	+
Flatten	Reshape only	? Fill the steps	+
FC 1	Fully connected + ReLU	? Fill the steps	+
FC 2	Fully connected + ReLU	? Fill the steps	+
Output	Fully connected + softmax	? Fill the steps	+

At Flatten, multiply the height, width and channel count to obtain the length of the vector.

04 / COUNT THE LEARNABLE VALUES

From one filter to the whole layer.

A convolutional filter spans the input channels. A fully connected neuron uses every input value. Both have one bias per output.

Convolution

$$N_{\text{params}} = (K_h \times K_w \times C_{\text{in}} + 1) \times C_{\text{out}}$$

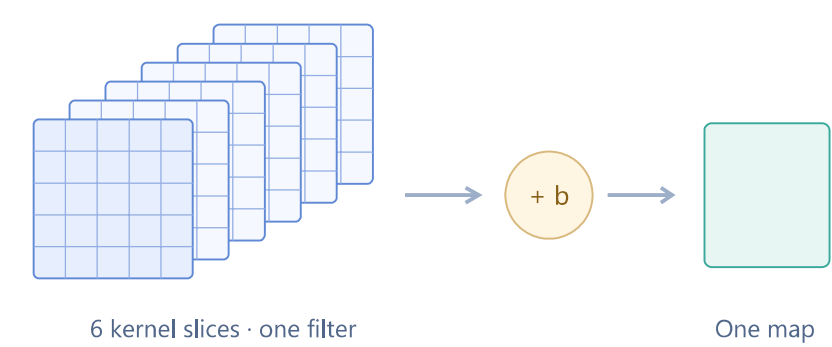
K_h and K_w : kernel height and width. C_{in} : input channels. C_{out} : filters.

Fully connected

$$N_{\text{params}} = (N_{\text{in}} + 1) \times N_{\text{out}}$$

N_{in} : input values. N_{out} : output neurons. The +1 accounts for one bias per output.

A FILTER HAS DEPTH, TOO



Why multiply by input channels?

One filter uses **25 weights per input channel**. With 6 input channels, it needs 6 sets of 25 weights, plus one bias.

$$N_{\text{one filter}} = (5 \times 5 \times 6) + 1 = 151$$

The bias is added once per filter.

Open each layer to count its weights, its biases, and then its total parameters.

LAYER	INPUT → OUTPUT	PARAMETERS
Conv 1	1 → 6 channels	? Fill the steps

Pool 1	2 × 2 max pool · stride 2 · padding 0	? Fill the steps	+
Conv 2	6 → 16 channels	? Fill the steps	+
Pool 2	2 × 2 max pool · stride 2 · padding 0	? Fill the steps	+
Flatten	Reshape only	? Fill the steps	+
FC 1	400 → 120 neurons	? Fill the steps	+
FC 2	120 → 84 neurons	? Fill the steps	+
Output	84 → 10 neurons	? Fill the steps	+

ReLU, max pooling, flatten and softmax have no trainable parameters.

► Reveal the parameter budget

Why does max pooling contribute zero parameters?

CHAPTER 2 / FOLLOW A PHOTOGRAPH

A colour photograph → 1,000 classes

AlexNet

Now follow a colour image through five convolutional layers. More channels and two large fully connected layers create a much larger parameter budget.

01 / SEE THE WHOLE NETWORK

Start with the big picture.

Read each row from left to right, then continue on the next row. Select a layer to inspect what it does.



Pixels become responses.

Keep the same input and step through the network. Every map below is an activation from a trained network, not a resized copy of the input.

Input image



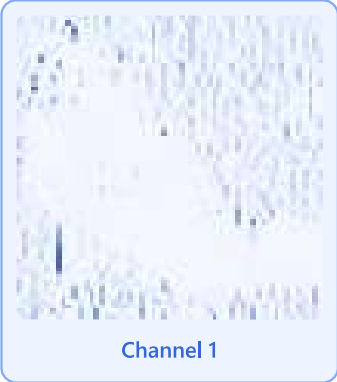
227 × 227 × 3

→

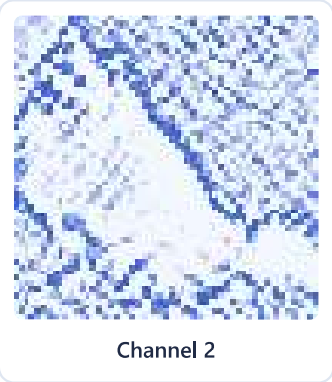
Conv 1 · feature maps

96 channels

Channel 1



Channel 2



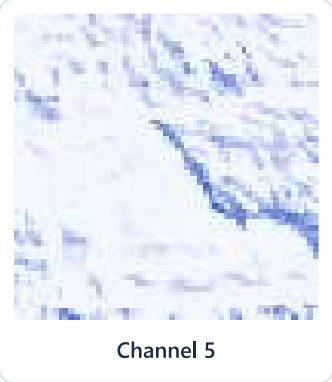
Channel 3



Channel 4



Channel 5



Channel 6





Channel 1 / 96

Actual size: 55 × 55

Maximum response: 112.854

Low response

High response

· Each map is scaled separately to reveal its pattern.

Conv 1: Each of the 96 filters spans every input channel. It slides across the spatial grid to produce one feature map. ReLU keeps positive responses and sets negative responses to zero. Local response normalization then rescales responses across nearby channels, without changing the shape or adding learned parameters.

A channel answers “where did this filter respond?” Early maps may highlight strokes, edges or contrast. Deeper maps combine earlier responses; they need not look like a recognizable picture.

Conv 3 and Conv 4 both produce 384 feature maps. Must their responses be identical?

03 / TRACE THE DIMENSIONS

Trace the dimensions of the image as it flows through the network.

Open a layer, substitute the given values, then fill in the intermediate results and final output shape. Dimensions are for one image.

Output =

Input − $K + 2(P)$

Stride

+ 1

K is the kernel size. P is the padding.

Conv 1	11 × 11 convolution · stride 4 · padding 0	? × ? × ? Fill the steps	+
Pool 1	3 × 3 max pool · stride 2 · padding 0	? × ? × ? Fill the steps	+
Conv 2	5 × 5 convolution · stride 1 · padding 2	? × ? × ? Fill the steps	+
Pool 2	3 × 3 max pool · stride 2 · padding 0	? × ? × ? Fill the steps	+
Conv 3	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Conv 4	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Conv 5	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Pool 3	3 × 3 max pool · stride 2 · padding 0	? × ? × ? Fill the steps	+
Flatten	Reshape only	? Fill the steps	+
FC 1	Fully connected + ReLU	? Fill the steps	+
FC 2	Fully connected + ReLU	? Fill the steps	+
Output	Fully connected + softmax	? Fill the steps	+

Notice Conv 3 → Conv 4 → Conv 5: padding preserves the spatial dimensions, while the number of filters determines the channel count.

04 / COUNT THE LEARNABLE VALUES

From one filter to the whole layer.

A convolutional filter spans the input channels. A fully connected neuron uses every input value. Both have one bias per output.

Convolution

$$N_{\text{params}} = (K_h \times K_w \times C_{\text{in}} + 1) \times C_{\text{out}}$$

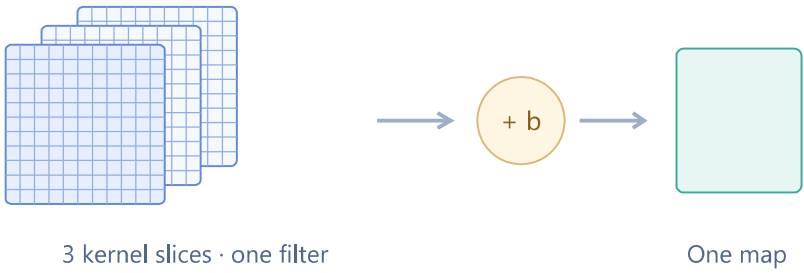
K_h and K_w : kernel height and width. C_{in} : input channels. C_{out} : filters.

Fully connected

$$N_{\text{params}} = (N_{\text{in}} + 1) \times N_{\text{out}}$$

N_{in} : input values. N_{out} : output neurons. The +1 accounts for one bias per output.

A FILTER HAS DEPTH, TOO



Why multiply by input channels?

One filter uses **121 weights per input channel**. With 3 input channels, it needs 3 sets of 121 weights, plus one bias.

$$N_{\text{one filter}} = (11 \times 11 \times 3) + 1 = 364$$

The bias is added once per filter.

Open each layer to count its weights, its biases, and then its total parameters.

LAYER	INPUT → OUTPUT	PARAMETERS
Conv 1	3 → 96 channels	? Fill the steps
Pool 1	3 × 3 max pool · stride 2 · padding 0	? Fill the steps
Conv 2	96 → 256 channels	? Fill the steps
Pool 2	3 × 3 max pool · stride 2 · padding 0	? Fill the steps
Conv 3	256 → 384 channels	? Fill the steps
Conv 4	384 → 384 channels	? Fill the steps
Conv 5	384 → 256 channels	? Fill the steps
Pool 3	3 × 3 max pool · stride 2 · padding 0	? Fill the steps
Flatten	Reshape only	? Fill the steps
FC 1	9,216 → 4,096 neurons	? Fill the steps
FC 2	4,096 → 4,096 neurons	? Fill the steps
Output	4,096 → 1,000 neurons	? Fill the steps

ReLU, max pooling, flatten and softmax have no trainable parameters. Response normalization and dropout also have none.

► Reveal the parameter budget

Which layer contributes the most trainable parameters?

CHAPTER 3 / FOLLOW A PHOTOGRAPH

Five blocks → 1,000 classes

VGG-16

Build a deeper network by repeating small convolutions. Five blocks extract features before three fully connected layers make a prediction.

01 / SEE THE WHOLE NETWORK

Start with the big picture.

Repeat a small filter

Every convolution uses a 3×3 kernel, followed by ReLU. A block contains two or three convolutions.

Keep the size, then halve it

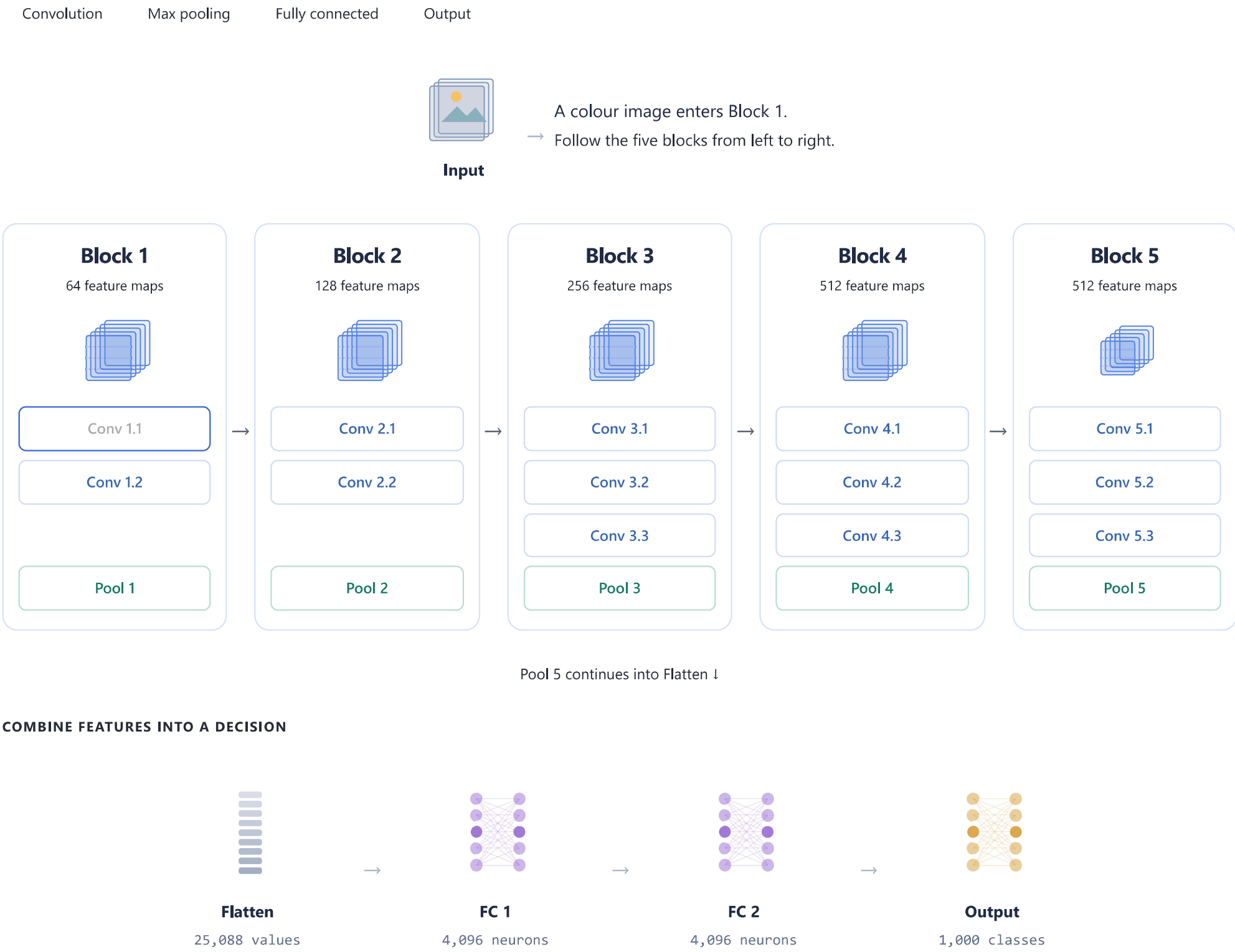
Stride 1 and padding 1 preserve the spatial size during convolution. Each block ends with 2×2 max pooling at stride 2.

Read the name

The 16 counts layers with weights: 13 convolutional layers and 3 fully connected layers.

Two consecutive 3×3 convolutions give a neuron access to a 5×5 region of the block’s input. Stacking small filters builds a larger receptive field, with a ReLU after each convolution.

Read each row from left to right, then continue on the next row. Select a layer to inspect what it does.



Input image



224 × 224 × 3

→

Conv 1.1 · feature maps

64 channels

Channel 1

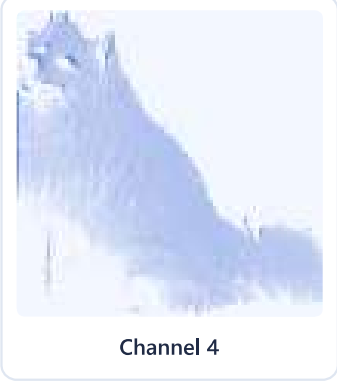
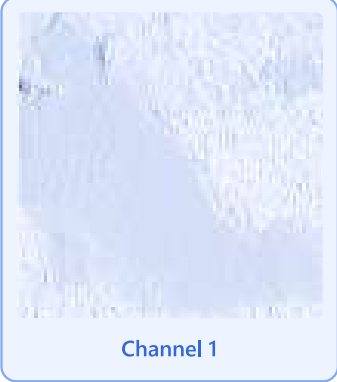
Channel 2

Channel 3

Channel 4

Channel 5

Channel 6



Showing 1–6 of 64. Use the channel slider to browse.

Low response

High response

· Each map is scaled separately to reveal its pattern.

Channel 1 / 64

Actual size: 224 × 224

Maximum response: 2.765

Conv 1.1: Each of the 64 filters spans every input channel. It slides across the spatial grid to produce one feature map. ReLU keeps positive responses and sets negative responses to zero.

A channel answers “where did this filter respond?” Early maps may highlight strokes, edges or contrast. Deeper maps combine earlier responses; they need not look like a recognizable picture.

Where does the image become spatially smaller in each VGG block?

03 / TRACE THE DIMENSIONS

Trace the dimensions of the image as it flows through the network.

Open a layer, substitute the given values, then fill in the intermediate results and final output shape. Dimensions are for one image.

Output =
$$\frac{\text{Input} - K + 2(P)}{\text{Stride}} + 1$$

K is the kernel size. P is the padding.

LAYER	OPERATION	OUTPUT SHAPE
Conv 1.1	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps +
Conv 1.2	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps +

https://kartikgupta98.github.io/dl-worksheets/worksheet10_architectures.html#lenet

13/17

Pool 1	2 × 2 max pool · stride 2 · padding 0	? × ? × ? Fill the steps	+
Conv 2.1	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Conv 2.2	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Pool 2	2 × 2 max pool · stride 2 · padding 0	? × ? × ? Fill the steps	+
Conv 3.1	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Conv 3.2	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Conv 3.3	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Pool 3	2 × 2 max pool · stride 2 · padding 0	? × ? × ? Fill the steps	+
Conv 4.1	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Conv 4.2	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Conv 4.3	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Pool 4	2 × 2 max pool · stride 2 · padding 0	? × ? × ? Fill the steps	+
Conv 5.1	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Conv 5.2	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Conv 5.3	3 × 3 convolution · stride 1 · padding 1	? × ? × ? Fill the steps	+
Pool 5	2 × 2 max pool · stride 2 · padding 0	? × ? × ? Fill the steps	+
Flatten	Reshape only	? Fill the steps	+
FC 1	Fully connected + ReLU	? Fill the steps	+
FC 2	Fully connected + ReLU	? Fill the steps	+
Output	Fully connected + softmax	? Fill the steps	+

Within each block, convolution preserves the spatial size. Pooling changes it through $224 \rightarrow 112 \rightarrow 56 \rightarrow 28 \rightarrow 14 \rightarrow 7$.

04 / COUNT THE LEARNABLE VALUES

From one filter to the whole layer.

A convolutional filter spans the input channels. A fully connected neuron uses every input value. Both have one bias per output.

Convolution

$$N_{\text{params}} = (K_h \times K_w \times C_{\text{in}} + 1) \times C_{\text{out}}$$

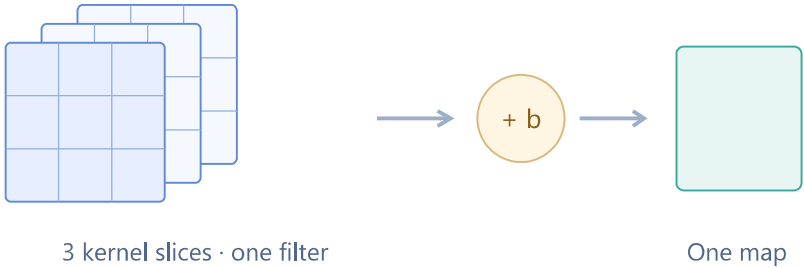
K_h and K_w : kernel height and width. C_{in} : input channels. C_{out} : filters.

Fully connected

$$N_{\text{params}} = (N_{\text{in}} + 1) \times N_{\text{out}}$$

N_{in} : input values. N_{out} : output neurons. The +1 accounts for one bias per output.

A FILTER HAS DEPTH, TOO



Why multiply by input channels?

One filter uses **9 weights per input channel**. With 3 input channels, it needs 3 sets of 9 weights, plus one bias.

$$N_{\text{one filter}} = (3 \times 3 \times 3) + 1 = 28$$

The bias is added once per filter.

Open each layer to count its weights, its biases, and then its total parameters.

LAYER	INPUT → OUTPUT	PARAMETERS	
Conv 1.1	3 → 64 channels	? Fill the steps	+
Conv 1.2	64 → 64 channels	? Fill the steps	+
Pool 1	2 × 2 max pool · stride 2 · padding 0	? Fill the steps	+
Conv 2.1	64 → 128 channels	? Fill the steps	+
Conv 2.2	128 → 128 channels	? Fill the steps	+
Pool 2	2 × 2 max pool · stride 2 · padding 0	? Fill the steps	+

Conv 3.1	128 → 256 channels	? Fill the steps	+
Conv 3.2	256 → 256 channels	? Fill the steps	+
Conv 3.3	256 → 256 channels	? Fill the steps	+
Pool 3	2 × 2 max pool · stride 2 · padding 0	? Fill the steps	+
Conv 4.1	256 → 512 channels	? Fill the steps	+
Conv 4.2	512 → 512 channels	? Fill the steps	+
Conv 4.3	512 → 512 channels	? Fill the steps	+
Pool 4	2 × 2 max pool · stride 2 · padding 0	? Fill the steps	+
Conv 5.1	512 → 512 channels	? Fill the steps	+
Conv 5.2	512 → 512 channels	? Fill the steps	+
Conv 5.3	512 → 512 channels	? Fill the steps	+
Pool 5	2 × 2 max pool · stride 2 · padding 0	? Fill the steps	+
Flatten	Reshape only	? Fill the steps	+
FC 1	25,088 → 4,096 neurons	? Fill the steps	+
FC 2	4,096 → 4,096 neurons	? Fill the steps	+
Output	4,096 → 1,000 neurons	? Fill the steps	+

ReLU, max pooling, flatten and softmax have no trainable parameters. Dropout also has no trainable parameters.

► Reveal the parameter budget

Conv 1.1 and Conv 1.2 both produce 64 channels. Why does Conv 1.2 have more parameters?

BRING THE THREE NETWORKS TOGETHER

Same building blocks. Different scale.

An activation belongs to a particular image. A parameter is reused every time the network processes an image.

LeNet 61,706 trainable parameters 32 × 32 × 1 input 2 convolutional layers · 3 fully connected layers 10 output classes	AlexNet 62,378,344 trainable parameters 227 × 227 × 3 input 5 convolutional layers · 3 fully connected layers 1,000 output classes	VGG-16 138,357,544 trainable parameters 224 × 224 × 3 input 13 convolutional layers · 3 fully connected layers 1,000 output classes
---	--	---

Double the batch size. What happens to the trainable parameter count?

▼ Three ideas to take into the next lecture

Spatial size and channel count do different jobs. Pooling reduces the spatial grid; convolution chooses how many feature maps to produce.

Activations and parameters are different quantities. A large output can be produced by a small set of shared weights.

Fully connected layers can dominate the parameter budget. Every input-to-output pair receives a separate weight.

Shapes use height × width × channels for one image. All parameter counts include biases. Built-in examples use recorded forward passes; your drawings are processed locally. Everything works offline.

[Revisit convolution basics](#) [LeNet reading](#) [AlexNet reading](#) [VGG reading](#)